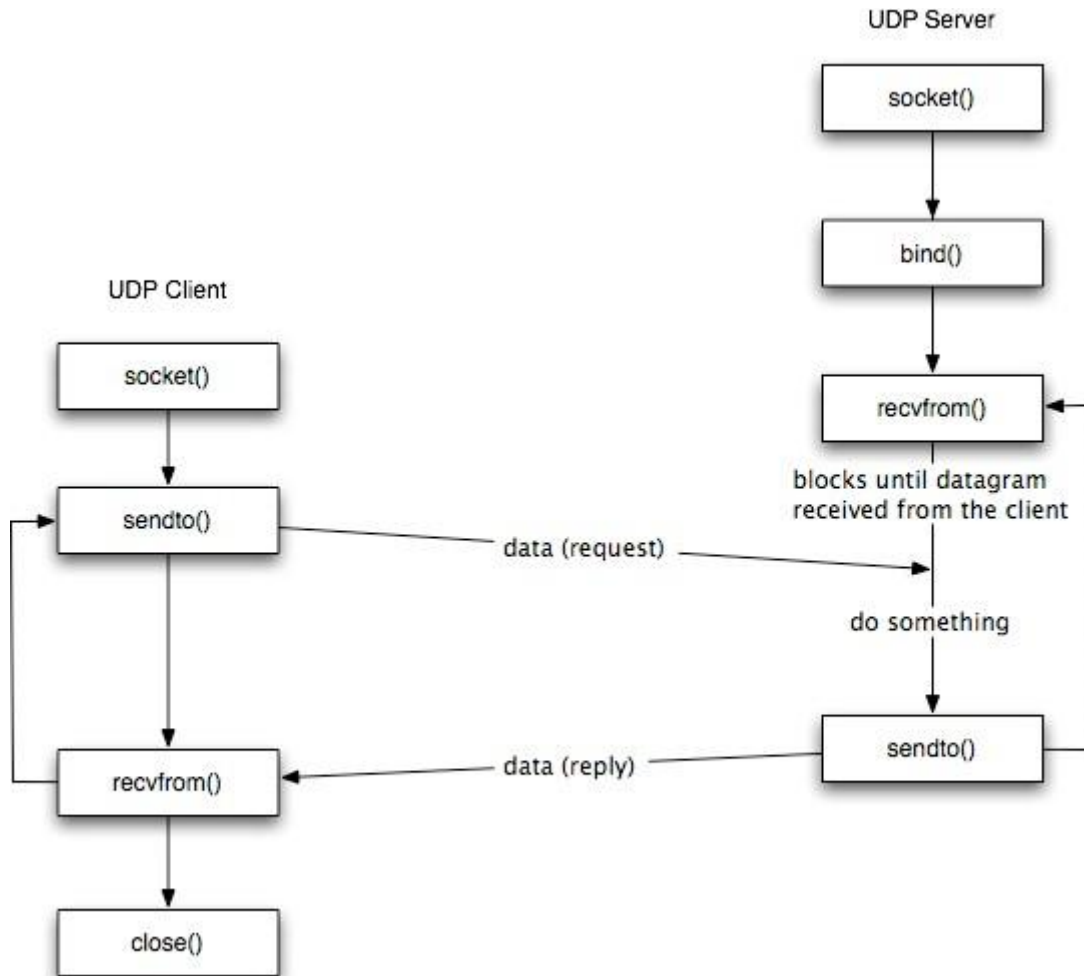


EXPERIMENT 2

AIM : To write a program to implement UDP



UDP Server :

1. Create a UDP socket.
2. Bind the socket to the server address.
3. Wait until the datagram packet arrives from the client.
4. Process the datagram packet and send a reply to the client.
5. Go back to Step 3.

UDP Client :

1. Create a UDP socket.
2. Send a message to the server.
3. Wait until response from the server is received.

4. Process reply and go back to step 2, if necessary.
5. Close socket descriptor and exit.

Necessary Functions :

`int socket(int domain, int type, int protocol)`

Creates an unbound socket in the specified domain.

Returns socket file descriptor.

Arguments :

domain – Specifies the communication

domain (AF_INET for IPv4/ AF_INET6 for IPv6)

type – Type of socket to be created

(SOCK_STREAM for TCP / SOCK_DGRAM for UDP)

protocol – Protocol to be used by the socket.

0 means use default protocol for the address family.

`int bind(int sockfd, const struct sockaddr *addr, socklen_t addrlen)`

Assigns address to the unbound socket.

Arguments :

sockfd – File descriptor of a socket to be bonded

addr – Structure in which address to be bound to is specified

addrlen – Size of *addr* structure

`ssize_t sendto(int sockfd, const void *buf, size_t len, int flags,`

`const struct sockaddr *dest_addr, socklen_t addrlen)`

Sends a message on the socket

Arguments :

sockfd – File descriptor of the socket

buf – Application buffer containing the data to be sent

len – Size of *buf* application buffer

flags – Bitwise OR of flags to modify socket behavior

dest_addr – Structure containing the address of the destination

addrlen – Size of *dest_addr* structure

`ssize_t recvfrom(int sockfd, void *buf, size_t len, int flags,`

`struct sockaddr *src_addr, socklen_t *addrlen)`

Receives a message from the socket.

Arguments :

sockfd – File descriptor of the socket

buf – Application buffer in which to receive data

len – Size of *buf* application buffer

flags – Bitwise OR of flags to modify socket behavior

src_addr – Structure containing source address is returned

addrlen – Variable in which size of *src_addr* structure is returned

int close(int fd)

Close a file descriptor

Arguments:

fd – File descriptor

In the below code, the exchange of one hello message between server and client is shown to demonstrate the model.

Implement client server communication using UDP Socket

UDP CLIENT

```
#include<stdio.h>
#include<string.h>
#include<sys/socket.h>
#include<stdlib.h>
#include<netdb.h>
int main()
{
    int clientsocket,port;
    struct sockaddr_in serveraddr,clientaddr;
    socklen_t len;
    char message[50];
    struct serveraddr;

    clientsocket=socket(AF_INET,SOCK_DGRAM,0);

    bzero((char*)&serveraddr,sizeof(serveraddr));
    len=sizeof(serveraddr);
    serveraddr.sin_family=AF_INET;
    printf("enter the port number ");
    scanf("%d",&port);
    serveraddr.sin_port=htons(port);
    printf("\nTrying to connect to the server.\n");

    printf("enter a message to server-\n");
    getchar()
    fgets(message,sizeof(message),stdin)

    sendto(clientsocket,message,sizeof(message),0,(struct
    sockaddr*)&serveraddr,sizeof(serveraddr));

    recvfrom(clientsocket,message,sizeof(message),0,(struct sockaddr*)&serveraddr,&len);
    printf("\nmessage from server:-\t%s\n",message);
    close(clientsocket);
}
```

UDP SERVER

```
#include<stdio.h>
#include<string.h>
#include<sys/socket.h>
#include<stdlib.h>
#include<netdb.h>
int main()
{
    int serversocket,clientsocket,port;
    struct sockaddr_in serveraddr,clientaddr;
    socklen_t len;
    char message[50];
    struct serveraddr;
    serversocket=socket(AF_INET,SOCK_DGRAM,0);

    bzero((char*)&serveraddr,sizeof(serveraddr));
    serveraddr.sin_family=AF_INET;
    printf("enter the port number ");
    scanf("%d",&port);
    serveraddr.sin_port=htons(port);
    serveraddr.sin_addr.s_addr=INADDR_ANY;
    bind(serversocket,(struct sockaddr*)&serveraddr,sizeof(serveraddr));

    bzero((char*)&clientaddr,sizeof(clientaddr));
    len=sizeof(clientaddr);
    printf("server running.....\nReading message from the client\n");

    recvfrom(serversocket,message,sizeof(message),0,(struct sockaddr*)&clientaddr,&len);
    printf("message from client:- %s",message);
    printf("\n enter a message to client:- \n");
    scanf("%s",message);
    sendto(serversocket,message,sizeof(message),0,(struct
sockaddr*)&clientaddr,sizeof(clientaddr));
    close(clientsocket);
    close(serversocket);
}
```